

1. Provider Availability Filtering — M2

Key Updates

1. `feature_engineering.py`:

- Updated `load_reference_data()` to read `provider_availability.csv` and include it in the returned dictionary under `"availability"`.
- Added `is_provider_available(provider_id, day_type, time_slot, availability_df)` to verify that a provider has active working hours covering the requested `day_type` (`weekday`, `weekend`, or specific day name) and `time_slot`.

2. `api.py`:

- In `/recommend`, candidates are filtered using `is_provider_available()` **before** calculating predictions or travel ETAs.
- Distinct 404 errors are returned:
 - *"No providers found for service_type: {service_type}"* when no providers exist in the catalog for that service.
 - *"Providers exist for service_type '{service_type}', but none are available for day_type '{day_type}' and time_slot '{time_slot}'."* when matching providers exist but none work during the requested window.

```
--- a/ml-service/feature_engineering.py
+++ b/ml-service/feature_engineering.py
@@ -23,6 +23,7 @@ def load_reference_data(data_dir=None):
    areas = pd.read_csv(f"{data_dir}/nairobi_areas.csv")
    traffic = pd.read_csv(f"{data_dir}/traffic_patterns.csv")
    providers = pd.read_csv(f"{data_dir}/service_providers.csv")
+   availability = pd.read_csv(f"{data_dir}/provider_availability.csv")

    # explode "A11|A12|A27" into one row per area_id for fast lookup
    traffic = traffic.assign(area_id=traffic["served_areas"].str.split("|")).explode("area_id")
-   return {"areas": areas, "traffic": traffic, "providers": providers}
+   return {"areas": areas, "traffic": traffic, "providers": providers, "availability": availability}
@@ -128,3 +129,66 @@
MODEL_FEATURES = ["distance_km", "congestion_multiplier", "time_slot",
                  "provider_area_road_quality", "provider_completion_rate"]

+
+TIME_SLOT_RANGES = {
+   "early_morning": ("05:00", "07:00"),
+   "morning_rush": ("07:00", "09:00"),
+   "mid_morning": ("09:00", "11:00"),
+   "midday": ("11:00", "14:00"),
```

```

+ "early_afternoon": ("14:00", "16:00"),
+ "evening_rush": ("16:00", "19:00"),
+ "evening": ("19:00", "21:00"),
+ "night": ("21:00", "05:00"),
+}
+
+WEEKDAYS = {"monday", "tuesday", "wednesday", "thursday", "friday"}
+WEEKENDS = {"saturday", "sunday"}
+
+
+def is_provider_available(provider_id, day_type, time_slot, availability_df):
+    """
+    Checks whether a given provider has an availability record covering the requested day_type
+    and time_slot.
+
+    Availability Policy for Missing Records:
+    Providers with no availability record in availability_df are treated as EXCLUDED (returns
+    False)
+    by default, ensuring unverified or unscheduled providers are not dispatched.
+    """
+    if availability_df is None or (isinstance(availability_df, pd.DataFrame) and
+    availability_df.empty):
+        return False
+
+    prov_records = availability_df.loc[availability_df["provider_id"].astype(str) == str(provider_id)]
+    if prov_records.empty:
+        return False
+
+    if "is_available" in prov_records.columns:
+        is_avail_mask = prov_records["is_available"].astype(str).str.lower().isin(["true", "1"])
+        prov_records = prov_records.loc[is_avail_mask]
+        if prov_records.empty:
+            return False
+
+    dtype = str(day_type).strip().lower()
+    if dtype == "weekday":
+        target_days = WEEKDAYS
+    elif dtype == "weekend":
+        target_days = WEEKENDS
+    elif dtype in WEEKDAYS or dtype in WEEKENDS:
+        target_days = {dtype}
+    else:
+        target_days = WEEKDAYS
+

```

```

+   day_matches = prov_records.loc[
+       prov_records["day_of_week"].astype(str).str.strip().str.lower().isin(target_days)
+   ]
+   if day_matches.empty:
+       return False
+
+   slot = str(time_slot).strip().lower()
+   if slot in ["weekend_day", "weekend"]:
+       slot = "midday"
+
+   if slot not in TIME_SLOT_RANGES:
+       return True
+
+   slot_start, slot_end = TIME_SLOT_RANGES[slot]
+
+   for _, row in day_matches.iterrows():
+       start_time = str(row["start_time"]).strip()
+       end_time = str(row["end_time"]).strip()
+
+       if slot_start < slot_end:
+           if start_time < slot_end and end_time > slot_start:
+               return True
+       else:
+           if start_time < slot_end or end_time > slot_start:
+               return True
+
+   return False

```

--- a/ml-service/api.py

+++ b/ml-service/api.py

@@ -19,7 +19,7 @@ from typing import Optional

import joblib

import pandas as pd

-from feature_engineering import load_reference_data, build_feature_row, MODEL_FEATURES

+from feature_engineering import load_reference_data, build_feature_row,

MODEL_FEATURES, is_provider_available

app = FastAPI(title="Nairobi Context-Aware Provider Reliability API", version="1.0")

@@ -107,6 +107,7 @@ def health():

"status": "ok",

"providers_loaded": len(ref["providers"]) if "providers" in ref else 0,

"areas_loaded": len(ref["areas"]) if "areas" in ref else 0,

+ "availability_loaded": len(ref["availability"]) if "availability" in ref else 0,

```

    }
@@ -183,10 +184,21 @@ def recommend(req: RecommendRequest):
    if candidates.empty:
        raise HTTPException(404, f"No providers found for service_type: {req.service_type}")

+ # Filter to available providers before scoring
+ availability_df = ref.get("availability")
+ available_mask = candidates["provider_id"].apply(
+     lambda pid: is_provider_available(pid, req.day_type, req.time_slot, availability_df)
+ )
+ available_candidates = candidates.loc[available_mask]
+
+ if available_candidates.empty:
+     raise HTTPException(
+         404,
+         f"Providers exist for service_type '{req.service_type}', but none are available for
day_type '{req.day_type}' and time_slot '{req.time_slot}'."
+     )
+
    ranked = []
- for _, provider_row in candidates.iterrows():
+ for _, provider_row in available_candidates.iterrows():
    try:
        score, features = _score_provider(req.client_area, provider_row, req.time_slot,
req.day_type)
    except Exception:

```

2. PostgreSQL + PostGIS Schema — M2

-- Enable PostGIS spatial extension

CREATE EXTENSION IF NOT EXISTS postgis;

-- -----

-- 1. nairobi_areas

-- Reference table for geographic areas in Nairobi with spatial centroids

-- -----

```

CREATE TABLE nairobi_areas (
    area_id VARCHAR(10) PRIMARY KEY,
    area_name VARCHAR(50) NOT NULL UNIQUE,
    geom GEOMETRY(Point, 4326) NOT NULL, -- PostGIS spatial column, SRID 4326
    area_type VARCHAR(30),
    avg_income VARCHAR(20),
    road_quality VARCHAR(20)
);

```

```
-- Spatial index for area centroid point geometry
CREATE INDEX idx_nairobi_areas_geom ON nairobi_areas USING GIST (geom);
```

```
-- -----
-- 2. clients
-- Registered clients requesting services
-- -----
```

```
CREATE TABLE clients (
  client_id VARCHAR(10) PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  area_id VARCHAR(10) REFERENCES nairobi_areas(area_id),
  phone VARCHAR(20),
  email VARCHAR(100)
);
```

```
CREATE INDEX idx_clients_area_id ON clients (area_id);
```

```
-- -----
-- 3. service_providers
-- Service providers and their base metadata
-- -----
```

```
CREATE TABLE service_providers (
  provider_id VARCHAR(10) PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  base_area_id VARCHAR(10) REFERENCES nairobi_areas(area_id),
  service_type VARCHAR(30) NOT NULL,
  rating DECIMAL(2,1),
  completion_rate DECIMAL(3,2),
  experience_years DECIMAL(4,1),
  avg_response_min DECIMAL(5,1),
  total_jobs INTEGER,
  is_verified BOOLEAN DEFAULT false,
  hourly_rate_ksh INTEGER
);
```

```
CREATE INDEX idx_service_providers_base_area_id ON service_providers (base_area_id);
```

```
-- -----
-- 4. traffic_patterns
-- Historical traffic corridor observations by time slot and day type
-- -----
```

```
CREATE TABLE traffic_patterns (
  traffic_pattern_id VARCHAR(10) PRIMARY KEY,
  corridor_name VARCHAR(50) NOT NULL,
```

```
direction VARCHAR(20),
time_slot VARCHAR(20) NOT NULL,
day_type VARCHAR(20) NOT NULL,
congestion_multiplier DECIMAL(3,2),
avg_speed_kmh DECIMAL(4,1),
congestion_level DECIMAL(3,1)
);
```

```
-- -----
-- 5. corridor_areas
-- Junction table: Many-to-Many relationship between traffic patterns & areas
-- -----
```

```
CREATE TABLE corridor_areas (
  corridor_area_id SERIAL PRIMARY KEY,
  traffic_pattern_id VARCHAR(10) REFERENCES traffic_patterns(traffic_pattern_id),
  area_id VARCHAR(10) REFERENCES nairobi_areas(area_id)
);
```

```
CREATE INDEX idx_corridor_areas_tp_id ON corridor_areas (traffic_pattern_id);
CREATE INDEX idx_corridor_areas_area_id ON corridor_areas (area_id);
```

```
-- -----
-- 6. provider_availability
-- Working hours and scheduled availability per provider
-- -----
```

```
CREATE TABLE provider_availability (
  availability_id SERIAL PRIMARY KEY,
  provider_id VARCHAR(10) REFERENCES service_providers(provider_id),
  day_of_week VARCHAR(10) NOT NULL,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL
);
```

```
CREATE INDEX idx_provider_availability_provider_id ON provider_availability (provider_id);
```

```
-- -----
-- 7. historical_bookings
-- Historical booking logs and arrival reliability records
-- -----
```

```
CREATE TABLE historical_bookings (
  booking_id SERIAL PRIMARY KEY,
  client_id VARCHAR(10) REFERENCES clients(client_id),
  provider_id VARCHAR(10) REFERENCES service_providers(provider_id),
  client_area_id VARCHAR(10) REFERENCES nairobi_areas(area_id),
```

```

    provider_area_id VARCHAR(10) REFERENCES nairobi_areas(area_id),
    service_type VARCHAR(30),
    distance_km DECIMAL(6,2),
    time_slot VARCHAR(20),
    day_type VARCHAR(20),
    congestion_multiplier DECIMAL(3,2),
    estimated_travel_min DECIMAL(6,1),
    arrival_reliability_score DECIMAL(4,3) CHECK (arrival_reliability_score BETWEEN 0 AND 1),
    on_time BOOLEAN,
    job_completed BOOLEAN,
    delay_min DECIMAL(6,1),
    actual_travel_min DECIMAL(6,1),
    client_rating_given DECIMAL(2,1)
);

```

```

CREATE INDEX idx_historical_bookings_client_id ON historical_bookings (client_id);
CREATE INDEX idx_historical_bookings_provider_id ON historical_bookings (provider_id);
CREATE INDEX idx_historical_bookings_client_area_id ON historical_bookings
(client_area_id);
CREATE INDEX idx_historical_bookings_provider_area_id ON historical_bookings
(provider_area_id);

```

3. Data Migration Script — M2

```

"""

```

migrate_csv_to_postgres.py

Migrates raw CSV files from data/raw/ into PostgreSQL/PostGIS tables defined in database/schema.sql.

Usage:

Dry-run validation mode (no database connection required):

```
python database/migrate_csv_to_postgres.py --dry-run
```

Live migration mode (reads connection from environment variables):

```
export DB_HOST=localhost
```

```
export DB_PORT=5432
```

```
export DB_NAME=nairobi_recommender
```

```
export DB_USER=postgres
```

```
export DB_PASSWORD=your_password
```

```
python database/migrate_csv_to_postgres.py
```

```

"""

```

```
import os
```

```
import sys
```

```
import argparse
```

```
from pathlib import Path
import pandas as pd
```

```
def get_db_connection_params():
    """Reads database connection parameters from environment variables."""
    return {
        "host": os.environ.get("DB_HOST", "localhost"),
        "port": int(os.environ.get("DB_PORT", 5432)),
        "dbname": os.environ.get("DB_NAME", "nairobi_recommender"),
        "user": os.environ.get("DB_USER", "postgres"),
        "password": os.environ.get("DB_PASSWORD", ""),
    }
```

```
def find_data_dir():
    """Locates the data/raw directory."""
    base = Path(__file__).resolve().parent
    candidates = [
        base.parent / "data" / "raw",
        base / "data" / "raw",
        Path("data/raw"),
        Path("../data/raw"),
    ]
    for c in candidates:
        if (c / "nairobi_areas.csv").exists():
            return c
    raise FileNotFoundError("Could not locate data/raw/ directory.")
```

```
# -----
# Migration functions for each table in dependency order
# -----
```

```
def migrate_nairobi_areas(conn, data_dir, dry_run=False):
    """
    1. nairobi_areas:
        Converts lat/lng columns into PostGIS geometry point:
        ST_SetSRID(ST_MakePoint(lng, lat), 4326)
    """
    df = pd.read_csv(data_dir / "nairobi_areas.csv")
    print(f"\n[1/7] nairobi_areas: Preparing {len(df)} records...")

    if dry_run:
```

```

        sample = df.head(2).to_dict(orient="records")
        print(f"    [DRY RUN] Would insert {len(df)} rows into nairobi_areas.")
        print(f"    [DRY RUN] Sample geom conversion:
ST_SetSRID(ST_MakePoint({sample[0]['lng']], {sample[0]['lat']}), 4326)")
        return len(df)

insert_sql = """
    INSERT INTO nairobi_areas (area_id, area_name, geom, area_type, avg_income,
road_quality)
    VALUES (%s, %s, ST_SetSRID(ST_MakePoint(%s, %s), 4326), %s, %s, %s)
    ON CONFLICT (area_id) DO NOTHING;
"""

cursor = conn.cursor()
inserted = 0
for _, row in df.iterrows():
    cursor.execute(
        insert_sql,
        (
            str(row["area_id"]).strip(),
            str(row["area_name"]).strip(),
            float(row["lng"]),
            float(row["lat"]),
            str(row["area_type"]).strip() if pd.notna(row["area_type"]) else None,
            str(row["avg_income"]).strip() if pd.notna(row["avg_income"]) else None,
            str(row["road_quality"]).strip() if pd.notna(row["road_quality"]) else None,
        ),
    )
    inserted += 1
conn.commit()
cursor.close()
print(f"    Successfully inserted {inserted} rows into nairobi_areas.")
return inserted

```

```

def migrate_clients(conn, data_dir, dry_run=False):
    """
    2. clients:
        References nairobi_areas(area_id).
    """
    df = pd.read_csv(data_dir / "clients.csv")
    print(f"\n[2/7] clients: Preparing {len(df)} records...")

    if dry_run:

```

```

print(f"    [DRY RUN] Would insert {len(df)} rows into clients.")
return len(df)

insert_sql = """
INSERT INTO clients (client_id, name, area_id, phone, email)
VALUES (%s, %s, %s, %s, %s)
ON CONFLICT (client_id) DO NOTHING;
"""

cursor = conn.cursor()
inserted = 0
for _, row in df.iterrows():
    phone_val = str(int(row["phone"])) if pd.notna(row["phone"]) and isinstance(row["phone"],
(int, float)) else str(row["phone"]) if pd.notna(row["phone"]) else None
    cursor.execute(
        insert_sql,
        (
            str(row["client_id"]).strip(),
            str(row["name"]).strip(),
            str(row["area_id"]).strip(),
            phone_val,
            str(row["email"]).strip() if ("email" in row and pd.notna(row["email"])) else None,
        ),
    )
    inserted += 1
conn.commit()
cursor.close()
print(f"    Successfully inserted {inserted} rows into clients.")
return inserted

def migrate_service_providers(conn, data_dir, dry_run=False):
    """
    3. service_providers:
        References nairobi_areas(area_id).
    """
    df = pd.read_csv(data_dir / "service_providers.csv")
    print(f"\n[3/7] service_providers: Preparing {len(df)} records...")

    if dry_run:
        print(f"    [DRY RUN] Would insert {len(df)} rows into service_providers.")
        return len(df)

    insert_sql = """

```

```

INSERT INTO service_providers (
    provider_id, name, base_area_id, service_type, rating,
    completion_rate, experience_years, avg_response_min,
    total_jobs, is_verified, hourly_rate_ksh
)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
ON CONFLICT (provider_id) DO NOTHING;
"""

cursor = conn.cursor()
inserted = 0
for _, row in df.iterrows():
    cursor.execute(
        insert_sql,
        (
            str(row["provider_id"]).strip(),
            str(row["name"]).strip(),
            str(row["base_area_id"]).strip(),
            str(row["service_type"]).strip(),
            float(row["rating"]) if pd.notna(row["rating"]) else None,
            float(row["completion_rate"]) if pd.notna(row["completion_rate"]) else None,
            float(row["years_experience"]) if pd.notna(row["years_experience"]) else None,
            float(row["avg_response_time_min"]) if pd.notna(row["avg_response_time_min"]) else
None,
            int(row["total_jobs_completed"]) if pd.notna(row["total_jobs_completed"]) else None,
            bool(row["is_verified"]) if pd.notna(row["is_verified"]) else False,
            int(row["hourly_rate_ksh"]) if pd.notna(row["hourly_rate_ksh"]) else None,
        ),
    )
    inserted += 1
conn.commit()
cursor.close()
print(f"    Successfully inserted {inserted} rows into service_providers.")
return inserted

def migrate_traffic_patterns(conn, data_dir, dry_run=False):
    """
    4. traffic_patterns:
        Loads corridor records (excluding pipe-delimited served_areas).
    """
    df = pd.read_csv(data_dir / "traffic_patterns.csv")
    print(f"\n[4/7] traffic_patterns: Preparing {len(df)} records...")

```

```

if dry_run:
    print(f"    [DRY RUN] Would insert {len(df)} rows into traffic_patterns.")
    return len(df)

```

```

insert_sql = """
INSERT INTO traffic_patterns (
    traffic_pattern_id, corridor_name, direction, time_slot,
    day_type, congestion_multiplier, avg_speed_kmh, congestion_level
)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
ON CONFLICT (traffic_pattern_id) DO NOTHING;
"""

```

```

cursor = conn.cursor()
inserted = 0
for _, row in df.iterrows():
    cursor.execute(
        insert_sql,
        (
            str(row["traffic_pattern_id"]).strip(),
            str(row["corridor_name"]).strip(),
            str(row["direction"]).strip() if pd.notna(row["direction"]) else None,
            str(row["time_slot"]).strip(),
            str(row["day_type"]).strip(),
            float(row["congestion_multiplier"]) if pd.notna(row["congestion_multiplier"]) else None,
            float(row["avg_speed_kmh"]) if pd.notna(row["avg_speed_kmh"]) else None,
            float(row["congestion_level"]) if pd.notna(row["congestion_level"]) else None,
        ),
    )
    inserted += 1
conn.commit()
cursor.close()
print(f"    Successfully inserted {inserted} rows into traffic_patterns.")
return inserted

```

```

def migrate_corridor_areas(conn, data_dir, dry_run=False):
    """

```

```

    5. corridor_areas:
        Junction table linking traffic_patterns and nairobi_areas.
        Splits served_areas string (e.g. 'A11|A12|A27') into one row per area.
    """

```

```

    df = pd.read_csv(data_dir / "traffic_patterns.csv")
    corridor_rows = []

```

```

for _, row in df.iterrows():
    tp_id = str(row["traffic_pattern_id"]).strip()
    served = str(row["served_areas"]).split("|") if pd.notna(row["served_areas"]) else []
    for area_id in served:
        area_id = area_id.strip()
        if area_id:
            corridor_rows.append((tp_id, area_id))

print(f"\n[5/7] corridor_areas: Prepared {len(corridor_rows)} junction mappings from
traffic_patterns.served_areas...")

if dry_run:
    print(f"    [DRY RUN] Would insert {len(corridor_rows)} rows into corridor_areas.")
    return len(corridor_rows)

insert_sql = """
    INSERT INTO corridor_areas (traffic_pattern_id, area_id)
    VALUES (%s, %s);
"""

cursor = conn.cursor()
inserted = 0
for tp_id, area_id in corridor_rows:
    cursor.execute(insert_sql, (tp_id, area_id))
    inserted += 1
conn.commit()
cursor.close()
print(f"    Successfully inserted {inserted} rows into corridor_areas.")
return inserted

def migrate_provider_availability(conn, data_dir, dry_run=False):
    """
    6. provider_availability:
        References service_providers(provider_id).
    """
    df = pd.read_csv(data_dir / "provider_availability.csv")
    print(f"\n[6/7] provider_availability: Preparing {len(df)} records...")

    if dry_run:
        print(f"    [DRY RUN] Would insert {len(df)} rows into provider_availability.")
        return len(df)

    insert_sql = """

```

```

INSERT INTO provider_availability (provider_id, day_of_week, start_time, end_time)
VALUES (%s, %s, %s, %s);
"""

```

```

cursor = conn.cursor()
inserted = 0
for _, row in df.iterrows():
    cursor.execute(
        insert_sql,
        (
            str(row["provider_id"]).strip(),
            str(row["day_of_week"]).strip(),
            str(row["start_time"]).strip(),
            str(row["end_time"]).strip(),
        ),
    )
    inserted += 1
conn.commit()
cursor.close()
print(f"    Successfully inserted {inserted} rows into provider_availability.")
return inserted

```

```

def migrate_historical_bookings(conn, data_dir, dry_run=False):
    """

```

```

    7. historical_bookings:

```

```

        References clients, service_providers, and nairobi_areas.

```

```

        Maps client_area and provider_area names to area_id.
    """

```

```

    df_areas = pd.read_csv(data_dir / "nairobi_areas.csv")
    area_name_to_id = {row["area_name"].strip().lower(): str(row["area_id"]).strip() for _, row in
df_areas.iterrows()}

```

```

    df = pd.read_csv(data_dir / "historical_bookings.csv")
    print(f"\n[7/7] historical_bookings: Preparing {len(df)} records...")

```

```

    if dry_run:

```

```

        print(f"    [DRY RUN] Would insert {len(df)} rows into historical_bookings.")
        return len(df)

```

```

    insert_sql = """

```

```

        INSERT INTO historical_bookings (
            client_id, provider_id, client_area_id, provider_area_id,
            service_type, distance_km, time_slot, day_type,

```

```

        congestion_multiplier, estimated_travel_min,
        arrival_reliability_score, on_time, job_completed,
        delay_min, actual_travel_min, client_rating_given
    )
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s);
"""

```

```

cursor = conn.cursor()
inserted = 0
for _, row in df.iterrows():
    client_area_str = str(row["client_area"]).strip().lower()
    prov_area_str = str(row["provider_area"]).strip().lower()

    client_area_id = area_name_to_id.get(client_area_str)
    prov_area_id = area_name_to_id.get(prov_area_str)

    cursor.execute(
        insert_sql,
        (
            str(row["client_id"]).strip(),
            str(row["provider_id"]).strip(),
            client_area_id,
            prov_area_id,
            str(row["service_type"]).strip() if pd.notna(row["service_type"]) else None,
            float(row["distance_km"]) if pd.notna(row["distance_km"]) else None,
            str(row["time_slot"]).strip() if pd.notna(row["time_slot"]) else None,
            str(row["day_type"]).strip() if pd.notna(row["day_type"]) else None,
            float(row["congestion_multiplier"]) if pd.notna(row["congestion_multiplier"]) else None,
            float(row["estimated_travel_min"]) if pd.notna(row["estimated_travel_min"]) else None,
            float(row["arrival_reliability_score"]) if pd.notna(row["arrival_reliability_score"]) else
None,
            bool(row["on_time"]) if pd.notna(row["on_time"]) else None,
            bool(row["job_completed"]) if pd.notna(row["job_completed"]) else None,
            float(row["delay_min"]) if pd.notna(row["delay_min"]) else None,
            float(row["actual_travel_min"]) if pd.notna(row["actual_travel_min"]) else None,
            float(row["client_rating_given"]) if pd.notna(row["client_rating_given"]) else None,
        ),
    )
    inserted += 1
conn.commit()
cursor.close()
print(f"    Successfully inserted {inserted} rows into historical_bookings.")
return inserted

```

```

# -----
# Main Migration Pipeline
# -----

def main():
    parser = argparse.ArgumentParser(description="Migrate raw CSVs to PostgreSQL with
PostGIS.")
    parser.add_argument(
        "--dry-run",
        action="store_true",
        help="Validate CSVs and display insertion counts without connecting to the database.",
    )
    args = parser.parse_args()

    data_dir = find_data_dir()
    print(f"Found CSV data directory at: {data_dir}")

    conn = None
    if not args.dry_run:
        try:
            import psycopg2
        except ImportError:
            print("Error: psycopg2 is not installed. Install it with `pip install psycopg2-binary` or run
with `--dry-run`.")
            sys.exit(1)

        db_params = get_db_connection_params()
        print(f"Connecting to PostgreSQL database '{db_params['dbname']}' on
{db_params['host']}:{db_params['port']}...")
        try:
            conn = psycopg2.connect(**db_params)
            print("Connected successfully.")
        except Exception as e:
            print(f"Database connection error: {e}")
            sys.exit(1)
    else:
        print("Running in DRY-RUN mode. No database operations will be executed.")

    tasks = [
        ("nairobi_areas", migrate_nairobi_areas),
        ("clients", migrate_clients),
        ("service_providers", migrate_service_providers),
        ("traffic_patterns", migrate_traffic_patterns),
    ]

```

```

        ("corridor_areas", migrate_corridor_areas),
        ("provider_availability", migrate_provider_availability),
        ("historical_bookings", migrate_historical_bookings),
    ]

    summary = {}
    for table_name, task_fn in tasks:
        try:
            count = task_fn(conn, data_dir, dry_run=args.dry_run)
            summary[table_name] = f"SUCCESS ({count} rows)"
        except Exception as e:
            if conn and not args.dry_run:
                conn.rollback()
            summary[table_name] = f"FAILED: {e}"
            print(f"    [ERROR] Failed migrating table '{table_name}': {e}")

    if conn:
        conn.close()

    print("\n" + "=" * 50)
    print("MIGRATION SUMMARY")
    print("=" * 50)
    for tbl, status in summary.items():
        print(f" {tbl:25s} -> {status}")
    print("=" * 50)

```

```

if __name__ == "__main__":
    main()

```

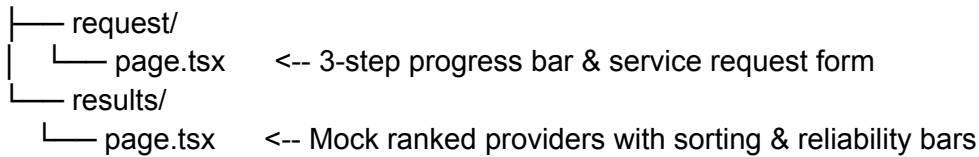
4. Scaffold Next.js + Mock-Data Prototype — M3

A Next.js 14 App Router application with TypeScript and Tailwind CSS has been scaffolded in **frontend**.

```

frontend/
├── package.json
├── tsconfig.json
├── next.config.mjs
├── postcss.config.js
├── tailwind.config.ts
├── app/
│   ├── globals.css
│   ├── layout.tsx
│   └── page.tsx

```



Key Page Implementations

1. `page.tsx`

- **3-Step Progress Indicator:** "Request Details" (Step 1 active), "View Recommendations" (Step 2), "Confirm Booking" (Step 3).
- **Form Controls:**
 - Service Type dropdown: Electrician, Plumber, Cleaner, Technician, Carpenter.
 - Client Area text input: e.g. "Kilimani".
 - Preferred Day radio group: Weekday (Mon - Fri) / Weekend (Sat - Sun).
 - Preferred Time Slot dropdown: Morning Rush (07:00 - 09:00), Midday (11:00 - 14:00), Evening Rush (16:00 - 19:00), Night (21:00 - 05:00), Weekend Day.
 - "Find Providers" submit button.

2. `page.tsx`

- **3-Step Progress Indicator:** Step 1 completed (check icon), Step 2 active ("View Recommendations"), Step 3 inactive.
- **Provider Cards:**
 - Hardcoded array of 5 mock provider objects with full context data.
 - Color-coded **Reliability Score** progress bar and percentage badge.
 - Contextual explanation text rendered in *italics* below the score.
 - ETA and distance rendered with inline SVG icons.
 - Hourly rate (KES / hr) and star rating (★).
 - "Select Provider" action button.
- **Client-Side Sorting:** Interactive sort dropdown supporting:
 - Reliability Score (High to Low)
 - Distance (Nearest First)
 - Price (Lowest First)
 - Rating (Highest First)

Configuration Files Created

- `package.json`: Configured with `next ^14.2.15`, `react ^18.3.1`, `tailwindcss ^3.4.13`, and TypeScript.
- `tsconfig.json`: Configured with path alias `@/*` pointing to `./*`.

- `tailwind.config.ts` & `postcss.config.js`: Configured for scanning `./app/**/*.{js,ts,jsx,tsx}`.
- `globals.css` & `layout.tsx`: Clean base layout with navigation header.

Ready for review. When you are ready, `npm install` and `npm run dev` can be executed in the `frontend` directory.